



Detecting and explaining structural changes in an evolving graph using a martingale[☆]

Shen-Shyang Ho^{ID*}, Tarun Teja Kairamkonda, Izhar Ali

Rowan University, Glassboro, 08028, NJ, USA

ARTICLE INFO

Keywords:

Conformal prediction
Martingale
Change detection
Shapley Additive Explanations (SHAP) Method
Explainable AI
Evolving graphs

ABSTRACT

Dynamic systems such as sensor networks, social networks, computer networks, and power grids can be modeled as evolving graphs, requiring effective methods to monitor structural changes in real-time. In this paper, we present a change detection framework for detecting global structural changes in evolving graphs. This framework combines multiple martingale-based change detectors using different graph features. We establish a mathematical relationship between an additive martingale (derived from the multiple graph features) and the Shapley Additive Explanations (SHAP) method, demonstrating that martingale values at detected change-points directly quantify each feature's contribution to the detected change. Thus, graph features with significantly high martingale values provide meaningful explanations for detected structural changes.

We demonstrate our approach using three synthetic graph types (random topology, scale-free, and small-world), showing how different graph features vary in detection performance across network types. This highlights the importance of using multiple graph features, as relying on any single feature might not detect certain changes. Additionally, we apply our approach to the MIT Reality dataset to identify changes in social interactions among dormitory students at some special time periods. Our results reveal which graph features best characterize the real-world network changes during these special time periods.

1. Introduction

Many real-world applications, such as sensor networks, social networks, computer networks, urban traffic networks, and power grids, can be modeled as groups of interacting entities represented as graphs evolving over time. It is important to monitor these evolving processes to detect significant local or global changes in interaction patterns. One particular task of interest is anomaly detection (global or local) [1], with the more generic related task being change-point detection [2]. For evolving graphs, a change can be understood as a *deviation from the regular pattern of the interacting entities* [3,4]. Formally, a change-point in an evolving graph is defined as a point in time at which the underlying distribution of graph features — characterizing nodes, edges, or local and global structural properties — undergoes a significant shift.

In this paper, we describe a novel martingale-based approach for detecting structural changes in evolving graphs. The use of multiple graph features in the martingale method allows identification of changes in specific graph properties related to structural changes. However, simply detecting changes is insufficient for many real-world applications—understanding why these changes occur is equally important. To address this explanatory need, we establish a connection between an

additive martingale change detection model and the Shapley values [5] from cooperative game theory. The Shapley values quantify feature contributions in a prediction model output but are computationally expensive to calculate exactly. They are typically approximated using the Shapley Additive Explanations (SHAP) method [6]. Our key contribution is demonstrating that the martingale value for each feature at detection time exactly equals its Shapley value when we use an additive martingale change detection approach based on multiple features. This result removes the need for costly approximations when quantifying feature contributions to detected changes. The direct relationship between the martingale values and the Shapley values provides practitioners with an immediate, computationally efficient method for explaining detected changes without additional approximation steps.

Our martingale-based explanation framework offers three key capabilities: (1) identifying which specific graph features are most sensitive to structural changes; (2) quantifying the relative importance of each feature at the time of detection; and (3) providing interpretable explanations for why a change was detected. Experimental results on both synthetic and real-world data reveal that features performing poorly on synthetic data can sometimes prove valuable in real-world contexts,

[☆] This document is the results of the research project funded by the National Science Foundation, United States.

* Corresponding author.

E-mail addresses: hos@rowan.edu (S.-S. Ho), kairam42@rowan.edu (T.T. Kairamkonda), aliizh94@students.rowan.edu (I. Ali).

highlighting the importance of diverse feature sets when analyzing complex network dynamics. This paper is an extension of [7] with an increased focus on explanation mechanisms and more extensive validation results.

The paper is organized as follows. Section 2 provides a brief review of existing change detection methods for evolving graphs and explanation techniques for graph-related tasks. Section 3 describes the probabilistic model for graph features representing evolving graphs over time and introduces the graph features used in our experiments. In Section 4, we formally define the change-point detection problem for evolving graphs and detail the martingale-based detection method. Section 5 explains how an additive martingale can be used to extract SHAP values for multiple graph features to quantify their impact on detected changes. Finally, in Section 6, we present and discuss experimental results on three types of synthetic evolving graphs, and demonstrate how our approach monitors and explains change-points in a real-world dataset.

2. Related work

Some well-established statistical approaches and monitoring techniques such as cumulative sum (CUSUM) control chart [8] and exponentially weighted moving average [9] were applied to detect changes in social networks [3]. Recent change-point detection approaches for evolving graphs aim to identify either local anomalies in node attribute values or global graph anomalies. Aggarwal et al. [10] proposed a thresholding method for detecting changes in node labels. Koutra et al. [11] framed graph change detection as a similarity assessment between two graphs based on node/edge characteristics. Flossdorf et al. [12] utilized multiple sets of metrics for online monitoring of evolving graphs. However, their detection performances are application dependent.

Grattarola et al. [13] proposed two statistical tests for change detection on evolving graphs using graph embeddings learned through an autoencoder to represent graph instances on constant-curvature Riemannian manifolds. Ferrari et al. [14] developed a non-parametric approach by monitoring nodes using graph-filtered signals encoding topology information. Huang et al. [15] utilized the spectrum of the Laplacian matrix to capture temporal relationships between graph snapshots. Xie et al. [16] proposed a multi-view feature interpretable change-point detection method based on a vector autoregressive model to track multiple targets. The approach finds multiple change-points in the graph structure using a single multidimensional vector encoding multiple feature time series. Kei et al. [17] developed a decoder-only latent space model for detecting change points in dynamic graphs, emphasizing efficient representation learning without requiring full graph reconstruction.

Recently, interest has grown in explainability and interpretability for concept drift and change detection in dynamic environments with streaming data [18], especially for manufacturing applications [19]. Existing graph-related explainability work includes methods for reasoning about link prediction on temporal knowledge graphs [20], and explaining predictions using graph neural networks [21].

Martingale-based detectors offer distribution-free change detection by testing data exchangeability, originating from conformal prediction frameworks [22]. Recent extensions include Nourtdinov et al. [23]'s conformal test martingales for hypergraphical dependency models. Tsaknaki et al. [24] proposed a Bayesian autoregressive online change-point detection method with time-varying parameters that enhances estimate accuracy and forecasting power through modeling temporal dependencies. For evolving graph structures, Sulem et al. [25] introduced a Siamese graph neural network for online change-point detection with adaptive capability. Hinder et al. [26] surveyed concept drift detection approaches in unsupervised data streams, highlighting the importance of not just detecting but also understanding changes. Our work bridges detection and explanation by formally connecting

martingale-based change detection with SHAP explanations for evolving graphs. To the best of our knowledge, this is the first approach that provides feature-based explanations for structural changes in evolving graphs.

3. Modeling feature distribution for evolving graphs

An evolving graph is represented by a sequence of static graph snapshots $(G_k)_{1 \leq k \leq n}$. Each graph snapshot is observed sequentially exactly once, from which we extract relevant graph features. We define the conditional density of a graph feature by

$$p_\theta(f(G_k) | f(G_{k-1}), \dots, f(G_1))$$

where $f : G \rightarrow F$ is a transformation that maps an input graph snapshot from G (the set of all possible graphs on a fixed set of vertices) to an output feature representation in feature space F . The feature representation $f(G_k) \in F = \mathbb{R}^{|V| \times d}$ creates a matrix with $|V|$ rows (one for each node in the fixed node set) and d columns (representing the feature dimension for each node) at time instance k .

The transformation f can represent various feature extraction or embedding methods, ranging from simple centrality measures to more computationally intensive methods such as Node2Vec [27], GraphSAGE [28], or dynamic graph embeddings [29]. In this paper, we focus on computationally efficient features for practical implementation. The graph features used in our empirical evaluation include:

1. **Degree Centrality (DC)** measures the connectivity of each node in the graph. For a node v , the degree centrality is defined as:

$$DC(v) = \frac{\deg(v)}{|V| - 1}$$

where $\deg(v)$ is the degree of node v (number of its connections), and the denominator normalizes the value to the range $[0, 1]$.

2. **Singular Value Decomposition (SVD) of the adjacency matrix** is derived by decomposing the adjacency matrix A , where A is an $n \times n$ matrix representing the graph with $A_{ij} = 1$ if there is an edge between nodes i and j , and $A_{ij} = 0$ otherwise. The SVD decomposes $A = U \Sigma V^T$ where U and V are orthogonal matrices and Σ is a diagonal matrix of singular values. The embedding is formed by taking the first d columns of $U \Sigma$, creating a low-dimensional representation such that nodes with structural similarity (e.g., same community, etc.) are positioned close together. We use $d = 2$ so that we can visualize the feature.
3. **Singular Values of Laplacian SVD (LSVD)** captures the spectrum of the graph Laplacian matrix. If $L = D - A$ is the Laplacian matrix, where D is the degree matrix (a diagonal matrix with $D_{ii} = \deg(i)$) and A is the adjacency matrix, then LSVD is the vector of singular values obtained from the SVD of L as $L = U_L \Sigma_L V_L^T$. The matrix Σ_L contains the singular values that encode fundamental graph properties such as connectivity patterns and community structure, as detailed in Section 3.2 of [15].

4. **Closeness Centrality (CC)** measures how quickly information can spread from a node to all other nodes. For a node v , closeness centrality is defined as:

$$CC(v) = \frac{|V| - 1}{\sum_{u \neq v} d(v, u)}$$

where $d(v, u)$ is the shortest path distance between nodes v and u . Higher values indicate nodes that can efficiently reach all other nodes in the network.

5. **Eigenvector (EV) Centrality** measures a node's influence based on the importance of its connections. If A is the adjacency matrix and x is the eigenvector centrality vector, then $Ax = \lambda x$ where λ is the largest eigenvalue of A , and x is the corresponding eigenvector. Each entry of x corresponds to a node's centrality score, with higher values indicating more influential nodes.

Unlike degree centrality, which only counts direct connections, eigenvector centrality recursively incorporates the importance of neighboring nodes, providing a more nuanced measure of centrality.

6. **Betweenness Centrality (BC)** quantifies how often a node lies on the shortest path between other nodes. For a node v , betweenness centrality is defined as:

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t , and $\sigma_{st}(v)$ is the number of those paths passing through node v . Nodes with high betweenness centrality act as bridges between different parts of the network and control information flow.

For the martingale-based change detection approach, rather than explicitly modeling the probability distribution for each graph feature, we compute a martingale value for each graph feature when a graph snapshot is observed and use this to identify structural changes in the evolving graph structure.

4. Structural change detection in an evolving graph using a martingale

The change detection problem involves identifying the time instance at which a deviation from the existing data generation model starts. We formally state our problem statement for an evolving graph similar to the problem statement in [2]:

Given a sequence of graph snapshots $(G_k)_{1 \leq k \leq n}$ with conditional density for a particular graph feature $p_\theta(f(G_k) | f(G_{k-1}), \dots, f(G_1))$. Before change occurs, the conditional density parameter $\theta = \theta_0$. After the change, $\theta = \theta_1$. The online problem is to “detect the occurrence of the change as soon as possible, with a fixed rate of false alarms” [2].

For our change-point detection problem, we extract multiple features from each snapshot of the evolving graph. For a graph snapshot G_k at time instance k , we apply m different feature extractors $\{f_1, f_2, \dots, f_m\}$, where each f_i captures a particular graph property as described in Section 3, to obtain feature values $\{f_1(G_k), f_2(G_k), \dots, f_m(G_k)\}$.

The martingale change detection approach was first proposed in [30] for labeled data streams and was later extended to regression models and unlabeled data streams [31]. Ho [30] utilized the idea that a lack of data exchangeability in observed streaming data implies a change in the underlying data distribution. We begin by reviewing the concept of a martingale.

Definition 1 ([31]). Given a sequence of random variables $\{M_i : 0 \leq i < \infty\}$. It is a **martingale** M with respect to the sequence $\{Z_i : 0 \leq i < \infty\}$ if for all $i \geq 0$ the following conditions hold:

- M_i is a measurable function of $Z_0, Z_1, \dots, Z_i$,
- $\mathbb{E}(|M_i|) < \infty$,
- $\mathbb{E}(M_{i+1} | Z_0, \dots, Z_i) = M_i$.

Vovk et al. [32] introduced the idea of testing exchangeability of streaming data using the **power martingale**, defined recursively as:

$$M_n = M_{n-1} \cdot \epsilon p_n^{\epsilon-1} \quad (1)$$

where $M_0 = 1$, and the term $\epsilon p_n^{\epsilon-1}$ is the betting function [33] that determines how to weight each new observation. The effectiveness of a power martingale with a fixed $\epsilon \in (0, 1)$ can vary significantly with different choices of ϵ as noted by [22]. In our work, we consider alternative betting functions to address this practical limitation.

A **betting function** $g : [0, 1] \rightarrow [0, \infty)$ such that $\int_0^1 g(p) dp = 1$ is a key component in a martingale-based change detection method [33] that transforms a sequence of p-values into a martingale sequence by specifying how to “bet” on each new p-value. Given a sequence of p-values, we construct a martingale sequence $\{M_n\}_{n=1}^\infty$ recursively as:

$$M_n = M_0 \prod_{i=1}^n g(p_i)$$

where $M_0 = 1$. When the p-values follow a uniform distribution under the null hypothesis (i.e., when no change has occurred), this sequence $\{M_n\}_{n=1}^\infty$ forms a martingale. Various betting functions [34] offer different sensitivity characteristics:

- **Mixture Power Martingale** averages multiple power martingales with different ϵ values—lower values place larger bets on small p-values, making the martingale more sensitive to potential changes, while values closer to 1 result in more conservative betting:

$$M_n = M_{n-1} \cdot \frac{1}{K} \sum_{k=1}^K \epsilon_k p_n^{\epsilon_k-1} \quad (2)$$

where $\{\epsilon_1, \epsilon_2, \dots, \epsilon_K\}$ are different ϵ parameter values and $M_0 = 1$. In this paper, we use $\epsilon \in [0.5, 0.9]$ to cover from highly sensitive to more conservative betting.

- **Beta betting** uses the Beta distribution density as the betting function:

$$M_n = M_{n-1} \cdot \text{Beta}(p_n; \alpha, \beta) \quad (3)$$

with shape parameters $\alpha, \beta > 0$ and initial value $M_0 = 1$. For $\alpha < 1$ and $\beta > 1$ (e.g., $\alpha = 0.5, \beta = 1.5$), it places higher bets on small p-values while maintaining a smooth betting profile across the entire range.

- **Plug-in Martingale** [35] replaces the fixed betting function $\epsilon p_i^{\epsilon-1}$ in (1) with an estimated probability density function based on previously observed p-values. This approach adapts to the specific characteristics of the data distribution.

Volkhonskiy et al. [33] further studied different variations of martingales for change detection, exploring various non-conformity measures and betting functions. To compute the non-conformity scores for each feature across all graph snapshots observed so far, we employ $\mathcal{K}$ -means clustering with $\mathcal{K} = 1$. These scores are then used to calculate p-values, which in turn are used to update the corresponding martingale value $M_i(t)$ at the current time instance $t = n$. Let $C_i((f_j(G_k))_{1 \leq k \leq n})$ be the cluster center for feature i . The non-conformity score for graph G_k is then defined as:

$$S_i(G_k) = \|f_i(G_k) - C_i((f_j(G_k))_{1 \leq k \leq n})\| \quad (4)$$

where $\|\cdot\|$ represents a distance measure between a feature vector and the cluster center.

The **non-conformity score** quantifies how much a data instance differs from other data instances [31]. The p-value p_n at time instance n is computed as:

$$p_n(\{(G_k)_{k=1}^n\}, \theta_n) = \frac{\#\{k : cs_k > cs_n\} + \theta_n \#\{k : cs_k = cs_n\}}{n} \quad (5)$$

where cs_k is the non-conformity score for the observation at time instance $k = 1, 2, \dots, n$ and θ_n is randomly chosen from $[0, 1]$ at time instance n to handle ties [32].

For the clustering and non-conformity score computation, we primarily use the **Euclidean distance**: $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$ due to its computational efficiency and intuitive geometric interpretation. For specific applications, we can use:

- **Mahalanobis distance:** $d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$ where $x, y \in \mathbb{R}^d$ are feature vectors and $\Sigma \in \mathbb{R}^{d \times d}$ is the sample covariance matrix computed from the observed feature vectors. This metric accounts for the correlation structure of data, making it useful for high-dimensional features with correlated components.
- **Cosine distance:** $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \cdot \|y\|}$ where $x, y \in \mathbb{R}^d$ are feature vectors, $x \cdot y$ denotes their dot product, and $\|x\|$ represents the Euclidean norm of vector x . This distance is used when the angle/orientation between feature vectors is more important than magnitude, useful for comparing graph embeddings.
- **Chebyshev distance:** $d(x, y) = \max_i |x_i - y_i|$ where $x, y \in \mathbb{R}^d$ are feature vectors and x_i, y_i represent their i th components. This metric captures the maximum difference across any dimension and is used when a change in any single feature component should be sufficient to signal a potential structural change.

According to Ville's inequality [36], for any non-negative martingale $\{M_k : k \geq 0\}$ with initial value $M_0 = 1$ and any $\lambda > 0$, we have

$$\mathbb{P}\left(\sup_{k \geq 0} M_k \geq \lambda\right) \leq \frac{1}{\lambda} \quad (6)$$

which serves as an upper bound for the probability of false alarm when no change-point exists. The choice of λ determines the acceptable risk of false alarms.

Fig. 1 illustrates our overall approach of monitoring an evolving graph using multiple features and their corresponding martingales. Algorithm 1 provides the complete procedure for processing a graph snapshot G_t at time t using multiple graph features simultaneously, maintaining a separate martingale for each feature.

The algorithm first initializes empty p -value lists as internal variables (Line 1). It then processes each graph feature in a loop (Line 2): extracting feature values (Line 3), computing non-conformity scores (Line 4), calculating p -values (Line 5), updating the p -value lists (Line 6), and computing the current martingale value (Line 7). For each feature, it determines if a change-point is detected by comparing the martingale value against threshold λ (Lines 8–12), resetting the values when a change is detected (Line 10). Finally, it returns the martingale values and change indicators (Line 14).

Fig. 2(a) compares different betting functions and distance metrics for change detection. In the top row, we observe that beta betting (left) shows detections at $(t = 89 - t = 92)$ while the mixture betting function (middle) performs better with the Mahalanobis distance detecting the change at exactly $t = 80$ (the true change-point). The power betting function (right) shows detection delays of 1–2 timesteps.

Fig. 2(b) examines each distance metric with different betting functions: Chebyshev distance produces the highest martingale peaks but with delayed detection; Cosine distance works well with power betting ($t = 82$) but varies with other betting functions; Euclidean distance shows moderate delays ($t = 81 - 90$), while Mahalanobis distance achieves the lowest detection delay ($t = 80 - 81$) when paired with mixture betting.

Different betting functions and distance metrics affect detection timing, but they rarely determine whether a change can be detected at all. As shown in Section 6.4.1, what truly matters is choosing the right graph features as some features completely fail to detect changes in certain graph types regardless of the betting or distance used.

5. Explaining detected changes in an evolving graph

The martingale values from Algorithm 1 contain information about which graph features were most sensitive to the detected change. In this section, we formalize our explanation mechanism using Shapley Additive Explanations (SHAP) method [6], establishing a direct link between the martingale-based change detection method and an interpretable model that identifies the most influential graph properties.

Algorithm 1 Monitoring Changes at time step t using Multiple Graph Features

Input: Graph G_t

Parameter: Feature extractors $\{f_1, f_2, \dots, f_m\}$, Threshold λ , Betting function b (default: power martingale with $\epsilon = 0.7$), Distance metric d (default: Euclidean)

Output: Martingale values $\{M_1(t), M_2(t), \dots, M_m(t)\}$, Change-Point indicators $\{c_1[t], c_2[t], \dots, c_m[t]\}$

- 1: Initialize empty p -value lists as internal variables $P_i = []$ for $i = 1, 2, \dots, m$
- 2: **for** each feature extractor f_i where $i = 1, 2, \dots, m$ **do**
- 3: Let $v_i(t) = f_i(G_t)$
- 4: Compute non-conformity scores for $(G_k)_{1 \leq k \leq t-1}$ and G_t using $v_i(k), k = 1, 2, \dots, t$ based on (4) with distance metric d
- 5: Compute the p -value $p_i(t)$ using non-conformity scores based on (5)
- 6: $P_i = \text{append}(P_i, p_i(t))$
- 7: Compute $M_i(t)$ using betting function b with p -values in P_i
- 8: **if** $M_i(t) \geq \lambda$ **then**
- 9: $c_i[t] = \text{Yes}$
- 10: RESET $M_i(t) = 1$ and $P_i = []$
- 11: **else**
- 12: $c_i[t] = \text{No}$
- 13: **end if**
- 14: **end for**
- 15: **return** $\{M_1(t), M_2(t), \dots, M_m(t)\}, \{c_1[t], c_2[t], \dots, c_m[t]\}$

5.1. Shapley value

The Shapley value was first proposed by Lloyd Shapley to calculate the expected marginal contribution of each player in cooperative game theory [5]. It takes into consideration how much the player affects the outcome of a model for every possible subset of players. The Shapley value for player i , denoted by ϕ_i , is calculated as

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} (f(S \cup \{i\}) - f(S)) \quad (7)$$

where N is the set of players, S is a subset of players excluding player i , and $f(X)$ is the model output (game result) given a set of players, X .

Shapley values possess several desirable properties that make them ideal for quantifying player contributions: (1) the sum of all Shapley values equals the total model output; (2) players with identical contributions receive identical Shapley values; (3) players that do not change the model output receive zero Shapley value; and (4) for combined games, the Shapley value equals the sum of the Shapley values for individual games. From (7), one can see that computing the exact Shapley values is computationally intensive when $|N|$ is large, as it requires evaluating all possible subsets of players.

5.2. SHAP method

Lundberg et al. [6] introduced the Shapley Additive Explanations (SHAP) method that quantifies the contribution of each feature to a prediction model—SHAP values are an approximation of the Shapley values for predictive models.

In the SHAP framework, the “players” in a game become “features” in a model, and the “game outcome” becomes the “model prediction”. Specifically, the set of players N in (7) is replaced by the set of features for a prediction model. The functions $f(S)$ and $f(S \cup \{i\})$ now represent prediction models $f_S(x)$ and $f_{(S \cup \{i\})}(x)$ trained on feature subsets S and $S \cup \{i\}$, respectively. The SHAP method provides a linear explanation model g with the following property for any prediction model f :

$$f(x) \approx g(x') = \phi_0 + \sum_{i \in S} \phi_i x'_i, \quad (8)$$

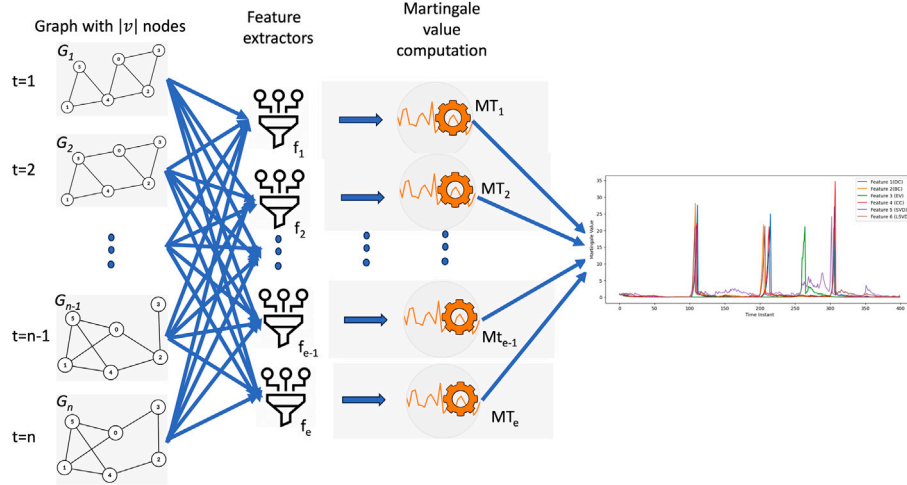


Fig. 1. Framework for detecting changes in evolving graphs. Left: A sequence of graph snapshots $G_1, G_2, \dots, G_n$ observed over time generated through the Erdos-Renyi model (see Section 6.1 for dataset generation description). Center: Parallel martingale tests ($MT_1, MT_2, \dots, MT_e$) monitoring different graph features. Right: Resulting martingale values for six graph features over time, with structural changes at $t = 101, 201$, and 301 . The varying heights of peaks indicate different feature sensitivities to structural changes.

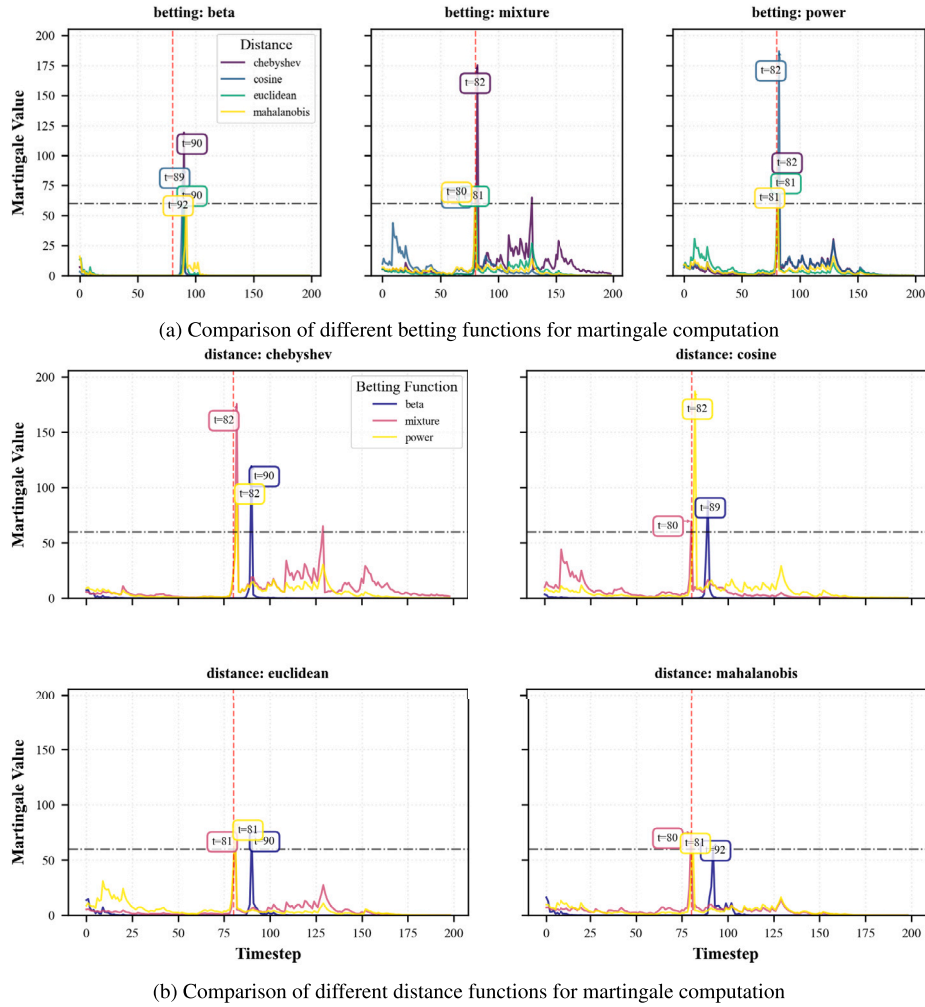


Fig. 2. Detection performance comparison of different betting and distance functions. The top row shows distance metrics across betting functions, while the bottom two rows show betting functions across distance metrics. The red vertical line at $t = 80$ indicates the actual change-point, and boxes show detection times. At each timestep, we extract the graph features (Section 3) from the Erdos-Renyi model (see Section 6.1 for dataset generation description), compute the martingale value for each feature, and add them together. Therefore, each curve shows the summation of individual feature martingales.

where $S \subseteq \{1, 2, \dots, m\}$ is a subset of the m available features, ϕ_0 represents the base value (expected prediction when no features are known), ϕ_i is the Shapley value representing the importance of feature i , and x' is a simplified binary vector where $x'_i = 1$ when feature i is present and $x'_i = 0$ when feature i is absent.

5.3. Additive martingale $M^A(t)$

For our change explanation problem, we can directly apply the SHAP linear explanation framework by modeling individual feature martingales as inputs to the linear model. This gives us a threshold classifier for change detection:

$$y = \mathbf{1}[f(S) > \tau] = \mathbf{1}\left[\sum_{i=1}^m M_i(t) > \tau\right] \quad (9)$$

where τ is the detection threshold, $\mathbf{1}[\cdot]$ is the indicator function, and $M_i(t)$ represents the martingale value for feature i at time t . The summation of the individual martingales (uniform weighting) allows us to treat each feature martingale as equally important for detection while still enabling attribution analysis.

Algorithm 2 Remove Line 8-13 in Algorithm 1 and add this after Line 14

```

1: if  $\sum_{i=1}^m M_i(t) \geq \lambda$  then
2:    $c[t] = \text{Yes}$ 
3:   RESET  $M_i(t) = 1$  and  $P_i = []$  for all  $i = 1, 2, \dots, m$ 
4: else
5:    $c[t] = \text{No}$ 
6: end if

```

To implement this change, we modify Algorithm 1 as follows: Remove lines 8–13 to disable individual feature martingale change detection. Before the return, add a check for change detection using the sum of all feature martingales and reset all martingales back to 1 if a change is detected, as shown in Algorithm 2.

Below, we prove that under certain assumptions, the summation of individual feature martingales, i.e., $\sum_{i=1}^m M_i(t)$ in our decision rule (9), also forms a martingale with the same property as (1). Recall the definition of a filtration, which represents the accumulation of information over time:

Definition 2. Let $(\Omega, \mathcal{F}, P)$ be a probability space and $F_0 = \{\emptyset, \Omega\}$, $F_1 \subseteq F_2 \subseteq \dots$ be a sequence of sub- σ -fields of $\mathcal{F}$. This sequence is called a **filtration** and is denoted by $(F_n)_{n \geq 0}$. A sequence of random variables $(M_n)_{n \geq 0}$ is *adapted* to the filtration if $M_n : \Omega \rightarrow \mathbb{R}$ is F_n -measurable for every $n \geq 0$. The sequence $(M_n)_{n \geq 0}$ is a **martingale** with respect to the filtration $(F_n)_{n \geq 0}$ if:

1. each M_n is measurable with respect to F_n (i.e., is adapted to the filtration),
2. $\mathbb{E}(|M_n|) < \infty$ for all $n \geq 0$, and
3. $\mathbb{E}(M_{n+1} | F_n) = M_n$ for all $n \geq 0$.

The initial value M_0 can be any F_0 -measurable integrable random variable, though for our betting functions and Ville's inequality applications (Eq. (6)), we specifically use $M_0 = 1$. In our evolving graph context, the filtration represents accumulated information from observed graph snapshots, and we monitor different graph features assuming they share a common information base.

Proposition 1. If $\{X_n\}_{n=0}^\infty$ and $\{Y_n\}_{n=0}^\infty$ are two martingales with respect to a filtration $\{F_n\}_{n=0}^\infty$, then the sequence $\{Z_n\}_{n=0}^\infty$ defined by

$$Z_n = X_n + Y_n$$

forms a martingale with respect to the same filtration.

Proof. (1) Since X_n and Y_n are each F_n -measurable, their sum $Z_n = X_n + Y_n$ is F_n -measurable. (2) The triangle inequality gives

$$\mathbb{E}[|Z_n|] = \mathbb{E}[|X_n + Y_n|] \leq \mathbb{E}[|X_n|] + \mathbb{E}[|Y_n|] < \infty,$$

since X_n and Y_n each have finite expectation of absolute value. For (3)

$$\begin{aligned} \mathbb{E}(Z_{n+1} | F_n) &= \mathbb{E}(X_{n+1} + Y_{n+1} | F_n) \\ &= \mathbb{E}(X_{n+1} | F_n) + \mathbb{E}(Y_{n+1} | F_n) \\ &= X_n + Y_n \quad (\text{by martingale property (3)}) \\ &= Z_n \end{aligned}$$

By mathematical induction, we can extend this result to any number of features:

Corollary 1. Let $\{X_n^{(1)}\}_{n=0}^\infty, \{X_n^{(2)}\}_{n=0}^\infty, \dots, \{X_n^{(m)}\}_{n=0}^\infty$ be m martingale sequences with respect to a fixed filtration $\{F_n\}_{n=0}^\infty$. Then the sequence $\{M_n^A\}_{n=0}^\infty$ defined by

$$M_n^A = \sum_{i=1}^m X_n^{(i)}$$

forms a martingale with respect to the same filtration.

Proposition 2. Let $\{X_n^{(1)}\}_{n=0}^\infty, \{X_n^{(2)}\}_{n=0}^\infty, \dots, \{X_n^{(m)}\}_{n=0}^\infty$ be m martingale sequences with respect to a fixed filtration $\{F_n\}_{n=0}^\infty$. Then the sequence $\{M_n^{avg}\}_{n=0}^\infty$ defined by

$$M_n^{avg} = \frac{1}{m} \sum_{i=1}^m X_n^{(i)}$$

forms a martingale with respect to the same filtration.

Proof.

$$\begin{aligned} \mathbb{E}(M_{n+1}^{avg} | F_n) &= \mathbb{E}\left(\frac{1}{m} \sum_{i=1}^m X_{n+1}^{(i)} \mid F_n\right) = \frac{1}{m} \mathbb{E}\left(\sum_{i=1}^m X_{n+1}^{(i)} \mid F_n\right) \\ &= \frac{1}{m} \sum_{i=1}^m \mathbb{E}(X_{n+1}^{(i)} | F_n) \\ &= \frac{1}{m} \sum_{i=1}^m X_n^{(i)} \quad (\text{by martingale property (3)}) \\ &= M_n^{avg} \end{aligned}$$

This additive martingale ensures that we have an initial martingale value of 1.

5.4. Relationship between SHAP values and martingale values

To understand the change detection and explanation process, let us examine how martingale values evolve from before the true change-point (CP) through detection.

Consider the above example of an Erdos–Renyi evolving graph monitored over 200 timesteps (see Section 6.1 for dataset generation description) where change detection is performed using the six graph features described in Section 3. We use mixture martingale (with ϵ ranging from 0.5 to 0.9) to track individual features, Mahalanobis distance metric, and compute an additive martingale with detection threshold $\lambda = 60.0$. The true structural change occurs at $t = 120$, and is detected at $t = 123$ where the additive martingale $M^A(123) = 67.44693$ exceeds $\lambda = 60.0$, triggering change detection with a delay of 3 timesteps as shown in Table 1 and the top graph in Fig. 3. We need to interpret which features contributed most significantly to this detection.

To explain which features contributed most to change detection, we need to compute Shapley values for each feature. In practice, these values are approximated using SHAP methods due to the computational complexity of exact calculation. However, before applying these approximation methods, we can theoretically analyze the additive nature

Table 1

Evolution of martingale values for six graph features before and after a structural change. The true change-point occurs at $t = 120$ (CP = 1), with detection triggered at $t = 123$ when the additive martingale $M^A(t)$ exceeds the threshold $\lambda = 60.0$.

t	CP	degree	betweenness	eigenvector	closeness	svd	lsvd	$M^A(t)$
115	0	0.05358	0.00001	0.00007	0.00080	0.35916	0.00559	0.41920
116	0	0.04845	0.00001	0.00008	0.00100	0.31759	0.00496	0.37208
117	0	0.03481	0.00001	0.00006	0.00073	0.32271	0.00360	0.36193
118	0	0.02702	0.00001	0.00005	0.00059	0.26763	0.00285	0.29815
119	0	0.08748	0.00007	0.00004	0.00236	0.92714	0.00942	1.02651
120	1	0.27275	0.00018	0.00003	0.00725	2.69683	0.02660	3.00363
121	0	0.87431	0.00052	0.00003	0.01922	7.71788	0.07094	8.68291
122	0	7.51733	0.00148	0.00002	0.04978	26.80217	0.20310	34.57389
123	0	14.44872	0.00282	0.00002	0.09600	52.47737	0.42200	67.44693
124	0	1.05456	0.78091	3.63507	0.89308	1.92083	0.70719	8.99164
125	0	0.82070	0.94075	3.39890	0.97631	1.55299	0.81404	8.50369
126	0	0.60147	0.70187	2.47767	0.68601	1.16397	0.58526	6.21623
127	0	0.55862	0.71460	1.73645	0.65601	1.10009	0.59179	5.35757
128	0	0.40748	0.57284	1.78518	0.51278	0.80595	0.63231	4.71654

of our model $f(S) = \sum_{i \in S} M_i$ in (9). In fact, we can show that the exact Shapley value for each feature i equals its martingale value M_i at a particular time instance by substituting $f(S) = \sum_{i \in S} M_i$ into (7):

$$\begin{aligned}
 \phi_i &= \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} (f(S \cup \{i\}) - f(S)) \\
 &= \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} \left[\left(\sum_{j \in S} M_j + M_i \right) - \sum_{j \in S} M_j \right] \\
 &= \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} \cdot M_i \\
 &= M_i
 \end{aligned}$$

since $\sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} = 1$.

To demonstrate this theoretical result, we applied *KernelExplainer* [6] to approximate Shapley values for our additive martingale model. The middle graph of Fig. 3 shows the SHAP values and their relationship with the actual martingale values. If the approximations were perfect, we would observe the square of the Pearson correlation coefficient, $R^2 = 1$ for all features. The resulting R^2 values instead range from 0.75 to 0.96 across features, confirming a strong linear relationship despite the inherent approximation errors introduced by the sampling-based *KernelExplainer* method. In our example, the features most responsible for change detection show the highest coefficients of determination: the degree feature achieves the highest R^2 value of 0.96, while SVD, which drives most of the detection signal, shows an $R^2 = 0.84$. Most importantly, at the detection point ($t = 123$), the SHAP value approximations show 96.1% agreement with martingale values for SVD (SHAP value of 50.45 vs. martingale value of 52.48) and 98.4% for degree (SHAP value of 14.22 vs. martingale value of 14.45).

Therefore, instead of computing SHAP value approximations at the cost of computational overhead and approximation errors, we can directly use each feature's martingale value at the detection point to calculate its percentage contribution to the total detection signal for our example:

$$\text{Contribution of } f_i = \frac{M_{f_i}(t_d)}{\sum_j M_{f_j}(t_d)} \times 100\% = \frac{M_{f_i}(123)}{67.44693} \times 100\%$$

where $M_{f_i}(t_d)$ is the martingale value of feature f_i at detection time $t_d = 123$.

Feature f_i	degree	betweenness	eigen-vector	close-ness	svd	lsvd
Martingale Value $M_{f_i}(123)$	14.449	0.003	0.00002	0.096	52.477	0.422
% Contribution	21.4%	0.0%	0.0%	0.1%	77.8%	0.6%

The percent contributions above are directly computed from individual martingale values (i.e., $M_{f_i}(t_d) / \sum_j M_{f_j}(t_d)$) and match exactly with the feature-wise contributions shown in the bottom graph of Fig. 3, where contributions were derived using the threshold-based classifier SHAP method with *KernelExplainer*. This confirms that our simplified contribution calculation method provides the same insights without the computational overhead of SHAP approximation techniques.

6. Experimental results

In this section, we present (i) empirical results on the change detection performance of multiple single feature martingales on evolving graph datasets generated using three graph generators to show that the feature types affect the performance of change detection on evolving graph of different characteristics and (ii) a real-world example of monitoring the martingale values derived from multiple feature types to explain the detected change. We first describe the datasets and graph generators used in our experiments (Section 6.1), followed by our evaluation measures (Section 6.2), experimental design (Section 6.3), and finally present and discuss our empirical results (Section 6.4).

6.1. Synthetic dataset descriptions

We generate sequences of graph snapshots using three random graph models — Erdos–Renyi [37], Barabasi–Albert [38], and Newman–Watts–Strogatz [39] — for our empirical evaluation. The random graphs are generated using the NetworkX package.¹

In the Erdos–Renyi model, edges between each pair of nodes are created independently with probability p for each snapshot. To introduce a change-point, we use probability p_{T_1} for the first T_1 time instances, then switch to probability p_{T_2} for the next T_2 time instances. The change-point occurs at time $T_1 + 1$. We quantify the difficulty of detecting this change using $\delta p = |p_{T_1} - p_{T_2}|$, where smaller values of δp signify a more difficult change-point detection problem.

The Barabasi–Albert model begins with an initial small star-shaped graph ($m + 1$ nodes, where $m \ll |V|$), incrementally adding new nodes that preferentially attach to existing nodes with higher degrees, until the desired number of nodes ($|V|$) is reached. m is defined as the number of edges to attach from a new node to existing nodes. For the first T_1 time instances, m is fixed at m_{T_1} . For the next T_2 time instances, $m = m_{T_2}$. The change-point occurs at $T_1 + 1$. We use $\delta m = |m_{T_1} - m_{T_2}|$ to quantify structural change difficulty. Smaller δm signifies more difficult change-point detection problem.

The Newman–Watts–Strogatz model generates small-world networks characterized by high clustering coefficients and short average path

¹ <https://networkx.org/documentation/stable/reference/generators.html>.

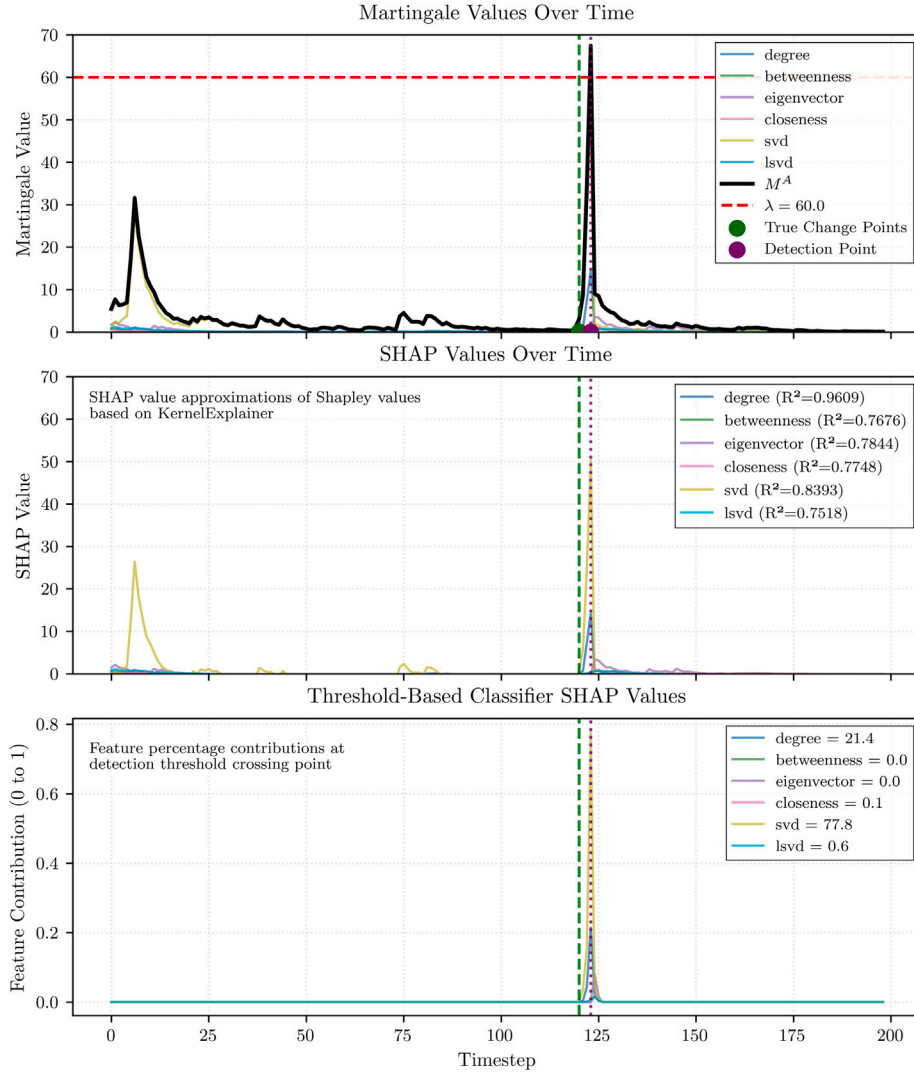


Fig. 3. Martingale and SHAP analysis for change detection. In our additive model, perfect approximations would yield $R^2 = 1$, but practical approximations achieve strong coefficients of determination (degree: 0.96, SVD: 0.84, others: 0.75–0.78), validating our theoretical proof that exact Shapley values equal martingale values.

lengths. Initially, nodes are arranged in a connected ring, each connected to their k nearest neighbors. Edges are then randomly rewired with probability p : this probability remains fixed at p_{T_1} for the first T_1 snapshots and then changes to p_{T_2} for the subsequent T_2 snapshots, with the change-point at $T_1 + 1$. The parameter $\delta p = |p_{T_1} - p_{T_2}|$ quantifies the structural change difficulty, with smaller values representing more subtle and thus challenging detection scenarios. The nearest neighbor parameter k , affecting local clustering and overall topology, remains constant across both time periods.

6.1.1. MIT reality dataset

The MIT Reality dataset [40] captures the evolution of social interactions among students in a dormitory based on their physical proximity. It records Bluetooth signals exchanged between students' mobile phones, indicating that senders and receivers were within approximately 10 meters at the time of communication. An edge is created between two users if their calculated proximity probability exceeds 0.3. All edges formed within the same day, regardless of the exact time, represent a daily snapshot of the dormitory's social network. We use 289 graph snapshots derived from 289 days of data, noting that some days contain missing records.

6.2. Evaluation measures

We evaluated detection performance using Mean Delay Time and the F_1 measure:

$$F_1 = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$$

where:

$$\begin{aligned} \text{Precision} &= \frac{\text{Number of Correctly Detected Change}}{\text{Number of Change Detected}} \\ &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \end{aligned}$$

$$\begin{aligned} \text{Recall} &= \frac{\text{Number of Correctly Detected Change}}{\text{Number of Changes}} \\ &= \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \end{aligned}$$

Precision measures the proportion of detected change-points that are true change-points identified within a specified window of t' time instances after the change-point occurs. Recall quantifies the proportion of actual change-points successfully detected by the algorithm within the specified detection window (t'). A change-point that the algorithm fails to detect within t' time instances is considered a *missed detection* or *false negative*. Conversely, a *false positive* occurs when the algorithm

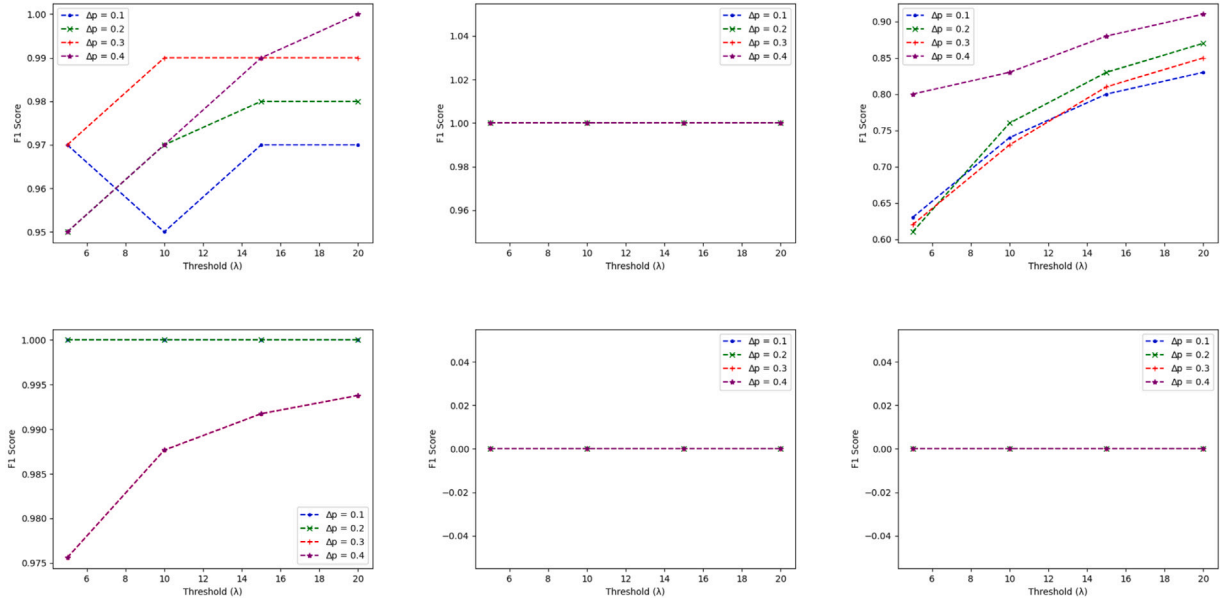


Fig. 4. Erdos-Renyi random graphs. F1-measure performance for change detection across six graph features: SVD (top-left), DC (top-middle), LSVD (top-right), CC (bottom-left), BC (bottom-middle), and EV (bottom-right). Results show detection performance across varying thresholds λ and structural change magnitudes δp . SVD and LSVD demonstrate the strongest detection capabilities, particularly for larger structural changes.

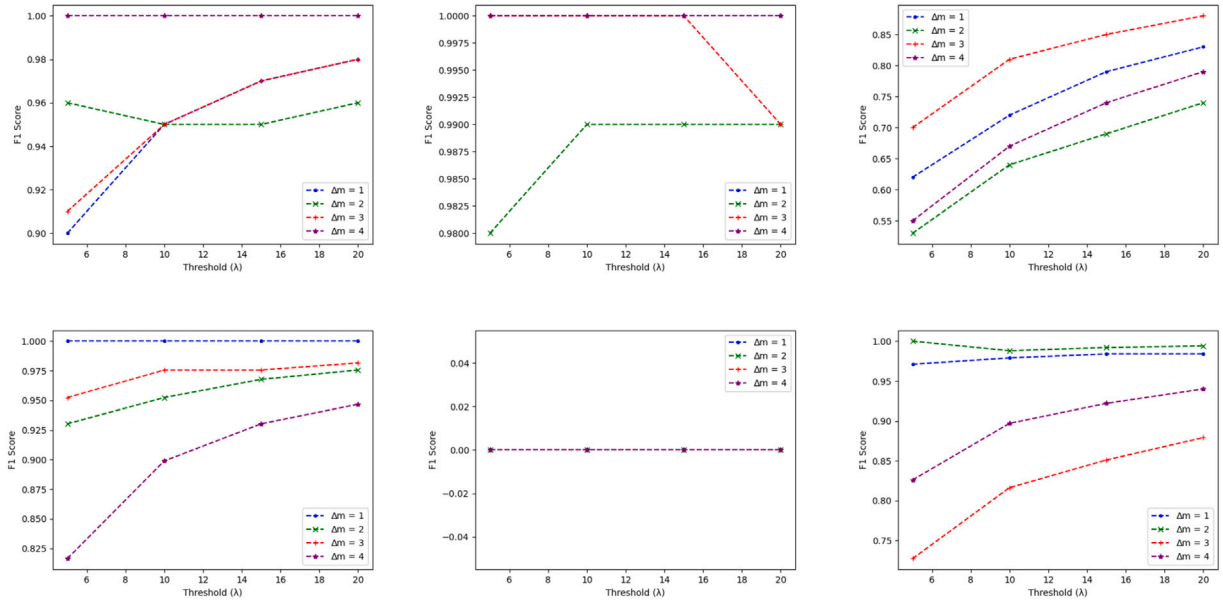


Fig. 5. Barabasi-Albert scale-free networks. F1-measure performance for change detection across six graph features: SVD (top-left), DC (top-middle), LSVD (top-right), CC (bottom-left), BC (bottom-middle), and EV (bottom-right). Results demonstrate how detection performance varies with threshold λ and attachment parameter changes δm . Features respond differently to structural changes, with SVD and DC showing the strongest overall performance.

incorrectly signals a change-point without an actual change occurring. F1 is the harmonic mean of precision and recall.

Mean Delay Time (MDT) for change detection is the sum of all *delay time* for all true positives divided by the number of true positives in a completed experiment.

6.3. Experimental design

Each experimental trial consists of a sequence of graph snapshots with a fixed number of nodes ($|V| = 30$), where the graph structure remains stable for $T_1 = 100$ time steps and then changes at the change-point ($t = 101$), followed by another $T_2 = 100$ time steps (total timesteps $T_1 + T_2 = 200$). We perform 20 trials for each combination of the

parameters: structural change magnitude (δp or δm), graph feature representation, and martingale threshold (λ), to analyze their effects on detection performance.

For feature representation, we use the graph features described in Section 3. We set threshold λ to 5, 10, 15, and 20, corresponding to progressively stricter false positive rate bounds from 20% down to 5%.

For the Erdos-Renyi model, we evaluate detection performance by varying the structural change magnitude parameter δp between 0.1 and 0.4. For the Barabasi-Albert model, we assess performance by varying δm (change in edges per new node) from 1 to 4. For the experiments on the Newman-Watts-Strogatz model, we vary the rewiring probability difference (δp) from 0.1 to 0.4, keeping the nearest neighbor parameter k fixed at 3 across all trials.

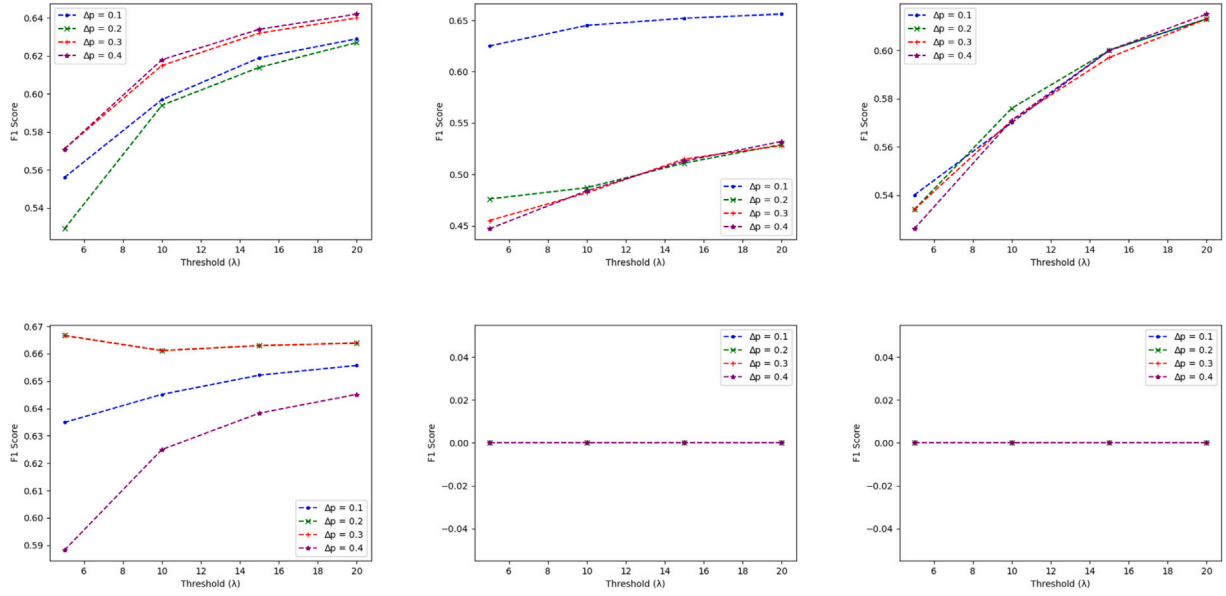


Fig. 6. Newman–Watts–Strogatz small-world networks. F1-measure performance for change detection across six graph features: SVD (top-left), DC (top-middle), LSVD (top-right), CC (bottom-left), BC (bottom-middle), and EV (bottom-right). Results illustrate detection performance variation with threshold λ and rewiring probability changes δp . Feature sensitivity varies significantly, with BC and EV showing minimal detection capability in this network type.

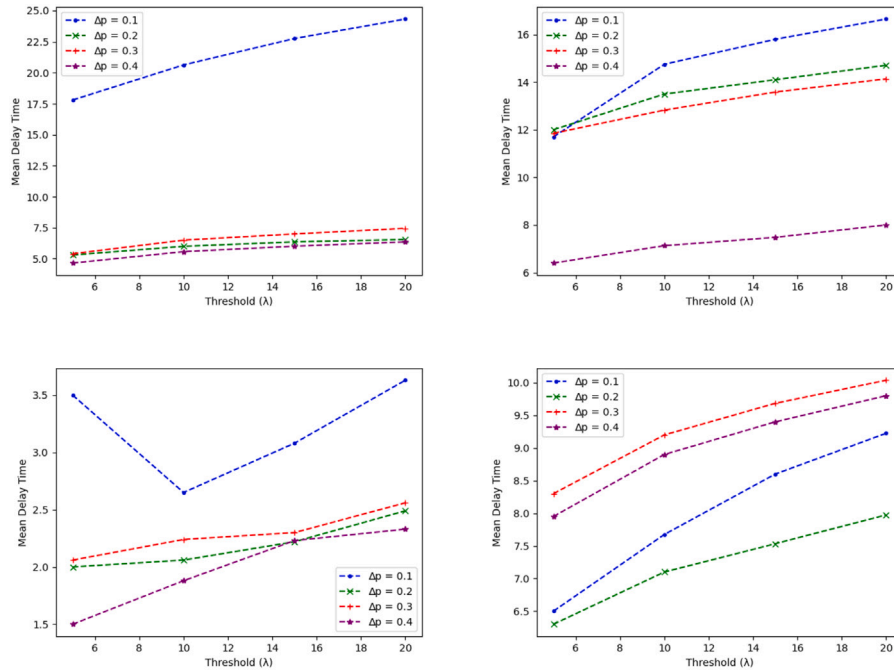


Fig. 7. Mean Delay Time (MDT) for change detection using the four effective graph features (TOP ROW: SVD (LEFT), DC (RIGHT); BOTTOM ROW: LSVD (LEFT), CC (RIGHT)) from Fig. 4 with varying detection threshold λ for different degree of change difficulty on the Erdős-Rényi type evolving graph.

6.4. Results and discussions

6.4.1. Effect of graph features and change difficulty on detection performance

For **Erdős-Rényi Random Graphs** (Fig. 4), martingale-based detection using SVD, DC, and CC features demonstrates consistently high performance (F1-score > 0.95). In contrast, BC and EV features exhibit poor detection performance, failing to effectively identify structural changes. The LSVD feature demonstrates variable performance that is dependent on both detection threshold and structural change difficulty, with significant improvements observed at higher thresholds and larger δp values.

For **Barabasi–Albert Scale-Free Networks** (Fig. 5), martingale-based detection using SVD and DC features provides robust performance (F1-score > 0.90) across different thresholds and structural change difficulties. BC proves ineffective for detecting structural changes in this graph model. The detection performance with LSVD, CC, and EV features exhibits significant dependence on the detection threshold (λ). However, no clear trend emerges linking increased change difficulty (δm) to reduced detection accuracy.

For **Newman–Watts–Strogatz Small-World Networks** (Fig. 6), structural changes prove the most challenging to detect among the three graph models considered in this study. BC and EV features fail to reliably detect structural changes in these graphs. Among the tested

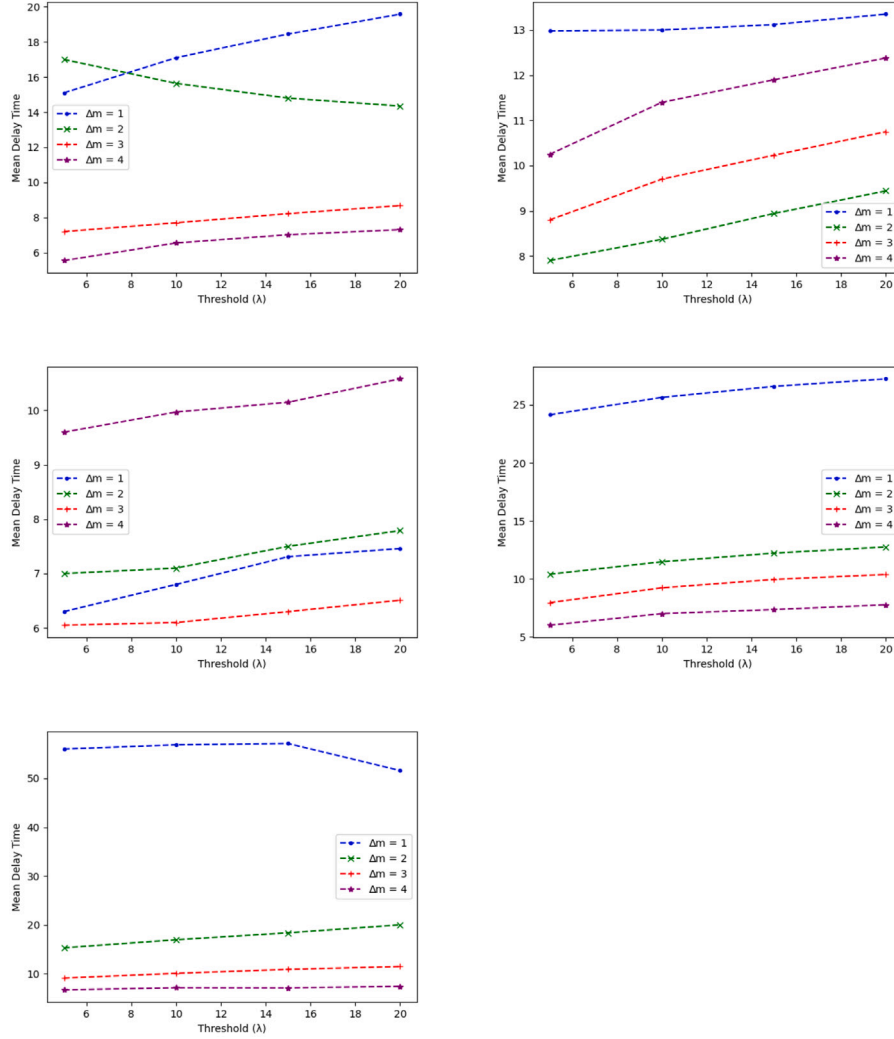


Fig. 8. Mean Delay Time (MDT) for change detection using the five effective graph features (TOP ROW: SVD (LEFT), DC (RIGHT); MIDDLE ROW: LSVD (LEFT), CC (RIGHT); BOTTOM ROW: EV) from Fig. 5 with varying detection threshold λ for different degree of change difficulty on the Barabási-Albert type evolving graph.

features, SVD, DC, LSVD, and CC perform relatively better, with F1-scores ranging from 0.5 to 0.68. However, the overall performance remains limited, indicating the need for identifying alternative, more effective features for small-world networks. The performance of SVD, DC, and LSVD depends heavily on detection threshold (λ), with no consistent trend observed in performance changes relative to increased structural change difficulty.

6.4.2. Effect of graph features and change difficulty on detection mean delay time

Figs. 7, 8, and 9 present Mean Delay Time (MDT) results for the effective graph features across the three network models, varying both detection thresholds and change magnitudes. BC and EV features are omitted from Figs. 7 and 9, and BC is omitted from Fig. 8, as these features demonstrated poor detection performance (shown in Figs. 4, 5, and 6).

Erdos-Rényi Random Graphs (Fig. 7): SVD, DC, and LSVD show expected behavior with longer detection times for subtle changes ($\delta p = 0.1$) and faster detection for pronounced changes ($\delta p = 0.4$). CC exhibits consistent MDT values across change magnitudes, warranting investigation. LSVD's lower MDT suggests it either detects changes very early or misses them entirely, potentially reducing reliability. Effective features (SVD, DC, CC) show MDT values of 1.5–25 time steps, with CC providing the best balance of high F1-scores and consistently low MDT (6.5–10) across all thresholds and change magnitudes.

Barabási-Albert Scale-Free Networks (Fig. 8): SVD, CC, and EV demonstrate intuitive behavior where smaller changes ($\delta m = 1$) require longer detection times than larger changes ($\delta m = 4$). LSVD and DC show more complex relationships between change magnitude and detection delay, requiring further analysis. DC offers the optimal combination of high detection accuracy (F1-score) and reasonable detection speed (MDT 8–13), making it most effective for Barabási-Albert networks across all thresholds and change magnitudes.

Newman-Watts-Strogatz Small-World Networks (Fig. 9): All four features display lower MDT values compared to other network types, but this correlates with lower F1-scores (Fig. 6), suggesting early detection or missed changes. Unlike other network types, MDT values show little correlation with change magnitude (δp), indicating detection timing may depend on factors beyond structural change magnitude. Combined F1-scores and MDT results confirm additional features are needed for reliable change detection in small-world networks.

A consistent pattern observed across all three network types is that MDT generally increases with higher detection thresholds (λ). This aligns with intuitive expectations, as reaching higher threshold values requires accumulating more evidence, leading to longer detection delays. This trade-off between false positive control (higher thresholds) and detection speed (lower MDT) represents a fundamental consideration in the practical application of martingale-based change detection.

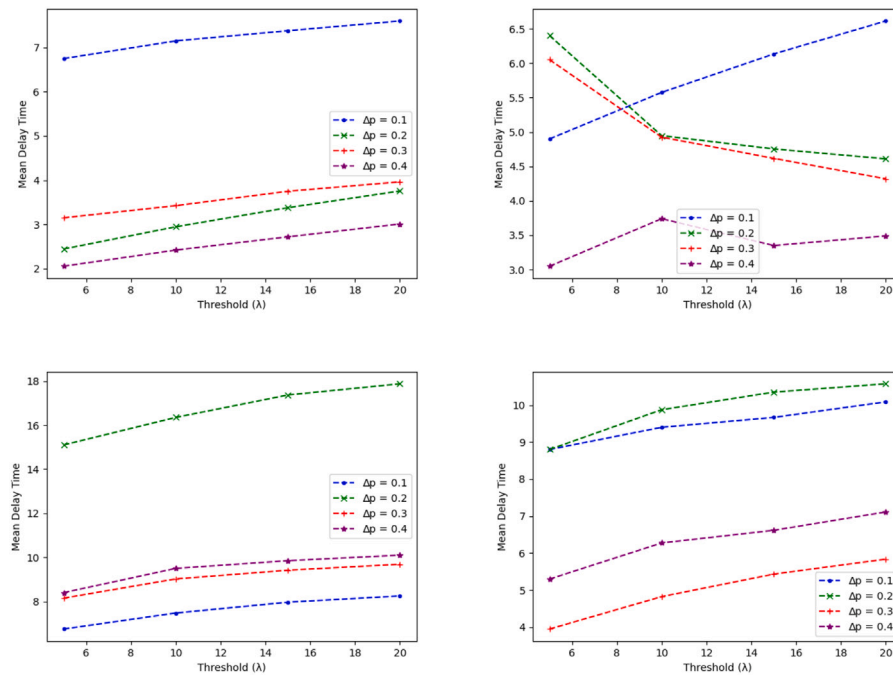


Fig. 9. Mean Delay Time (MDT) for change detection using the four effective graph features (TOP ROW: SVD (LEFT), DC (RIGHT); BOTTOM ROW: LSVD (LEFT), CC (RIGHT)) from Fig. 6 with varying detection threshold λ for different degree of change difficulty on the Newman–Watts–Strogatz small-world networks.

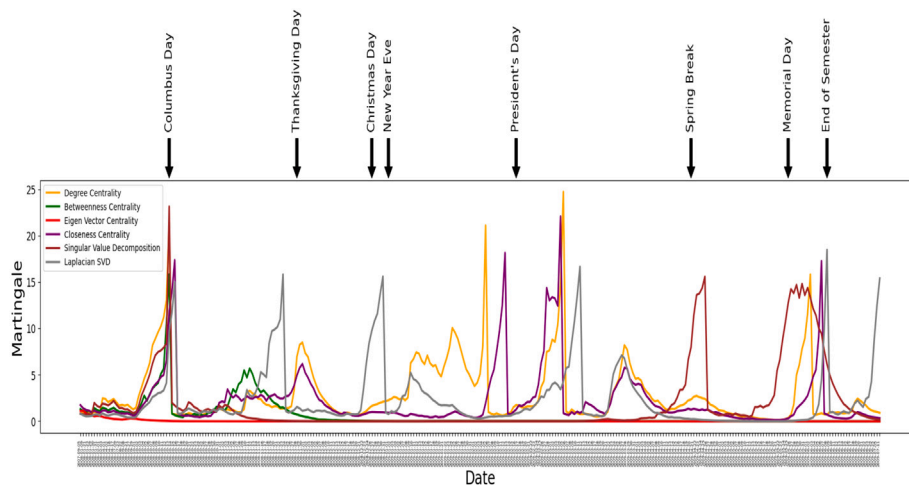


Fig. 10. Monitoring martingale values of six graph features on the social network evolution in a student dormitory over 2008 and early 2009 from the MIT reality dataset, with $\lambda = 10$.

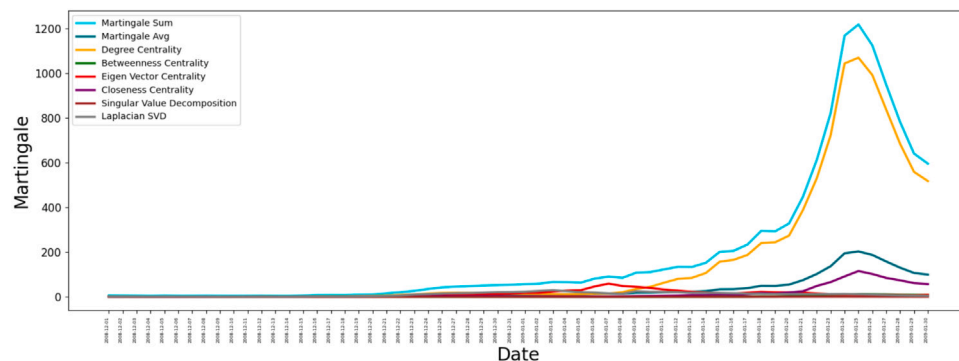


Fig. 11. Martingale values of the six single feature martingales and two additive martingales (Martingale Sum and Martingale Avg) on the MIT Reality dataset during the Christmas-New Year period. The sharp increase in values, particularly for Degree Centrality and Martingale Sum, indicates significant structural changes in social interactions.

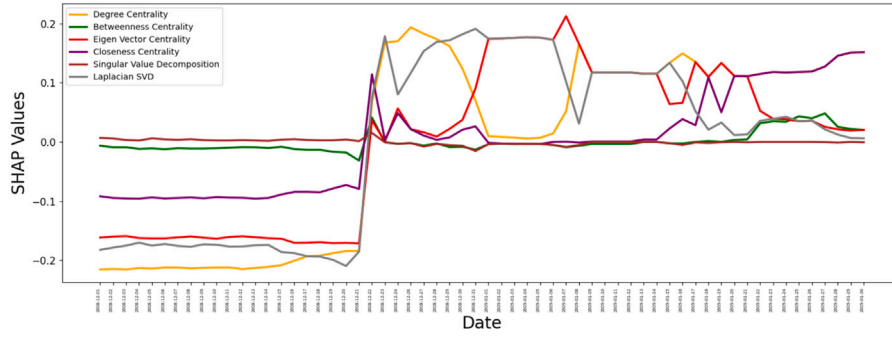


Fig. 12. SHAP values of the six graph features (Degree Centrality, Betweenness Centrality, Eigen Vector Centrality, Closeness Centrality, Singular Value Decomposition, and Laplacian SVD) used in the additive martingales on the MIT Reality dataset during the Christmas-New Year period. The abrupt changes in SHAP values highlight which features contributed most to detecting structural changes in the network.

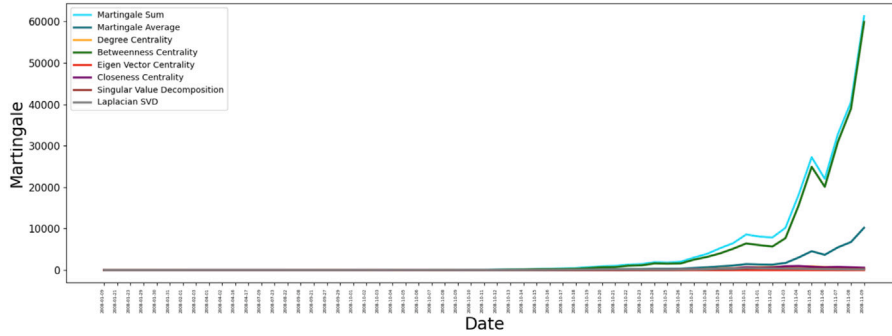


Fig. 13. Martingale values of the six single feature martingales and two additive martingales (Martingale Sum and Martingale Average) on the MIT Reality dataset during the Columbus Day period. Betweenness Centrality shows a dramatic increase, suggesting significant changes in network path structures during this period.

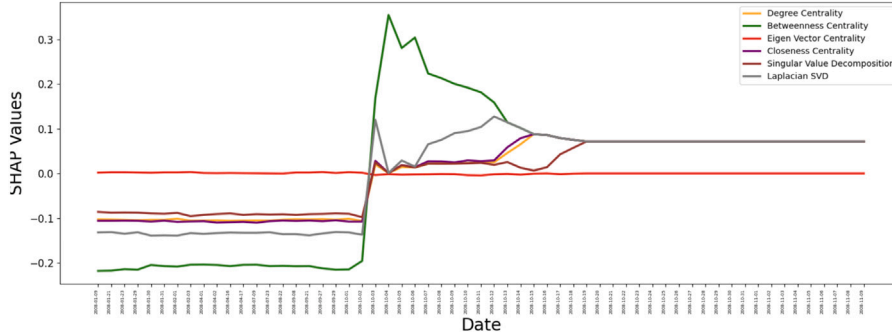


Fig. 14. SHAP values of the six graph features used in the additive martingale on the MIT Reality dataset during the Columbus Day period. The high spike in Betweenness Centrality SHAP values indicates that this feature was most influential in detecting the network structural change.

6.4.3. Monitoring martingale values with predefined threshold and explaining detected changes in evolving graphs

Fig. 10 illustrates martingale values for six graph features tracking social network evolution in a student dormitory over 289 days [40] with $\lambda = 10$. Notable martingale peaks across certain features correlate with holiday periods, suggesting potential seasonal patterns in social interactions.

Fig. 11 displays martingale values for both single feature and additive martingales during the Christmas-New Year period. The evolving graph is constructed from mobile phone proximity data (signals indicating devices within 10 m of each other), where structural changes reflect significant shifts in interpersonal proximity patterns. Fig. 12 shows the corresponding SHAP values for the six graph features during this period. Since martingale values correlate with SHAP values at change-point, we can attribute the structural changes primarily to shifts in LSVD, DC, and CC features.

For the Columbus Day period (Figs. 13 and 14), the structural change appears predominantly driven by changes in the BC feature.

While BC performed poorly in our synthetic data experiments, it emerges as the most significant feature for detecting change around Columbus Day. This suggests that the MIT Reality dataset exhibits structural characteristics distinct from our synthetic evolving graph models, highlighting the importance of feature diversity when analyzing real-world network dynamics.

7. Conclusions and future work

In this paper, we present a novel martingale-based approach for detecting structural changes in evolving graphs using multiple graph features. We provide a detailed framework explaining how martingale values from various graph features can be used to identify contribution patterns of each feature to detected change-points. Our key theoretical contribution is proving that in our additive martingale model, the exact Shapley value for each feature equals its martingale value ($\phi_i = M_i$), establishing a fundamental connection between martingale-based change detection model and explainable AI model.

Our findings demonstrate that this mathematical equivalence between martingale values and SHAP values eliminates the need for computationally expensive approximation methods, enabling direct identification and quantification of feature(s) that triggers a change detection. Through comprehensive experimentation with three synthetic evolving graph types (random topology, scale-free, and small-world networks) and six graph features, we empirically establish that while the selection of graph features significantly impacts detection capability, the specific combinations of distance metrics and betting functions primarily affect detection delay rather than detection capability. When applied to the MIT Reality dataset, our approach successfully identified and explained structural changes in social networks during holiday periods, with different features capturing distinct aspects of network transformation.

While this work focuses on detecting and explaining structural global changes in evolving graphs, future research will extend this method in two key directions: (1) detecting and explaining local structural changes in evolving graphs, which would provide more granular insights into network dynamics, and (2) reducing detection mean delay time by integrating a forecasting module into the evolving graph change-detection framework, which would enhance the practical utility of our approach for time-sensitive applications.

CRedit authorship contribution statement

Shen-Shyang Ho: Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Tarun Teja Kairamkonda:** Writing – review & editing, Visualization, Validation, Investigation, Data curation. **Izhar Ali:** Writing – review & editing, Software, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Tarun Kairamkonda reports financial support was provided by NSF. Shen-Shyang Ho reports financial support was provided by NSF. Izhar Ali reports financial support was provided by NSF. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is supported by NSF Award No.: 2213658.

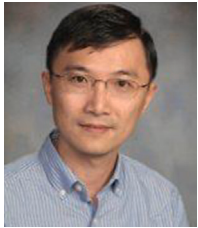
Data availability

Data will be made available on request.

References

- [1] L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, A. Liotta, Smart anomaly detection in sensor systems: A multi-perspective review, *Inf. Fusion* 67 (2021) 64–79.
- [2] M. Basseville, I.V. Nikiforov, et al., *Detection of Abrupt Changes: Theory and Application*, vol. 104, Prentice Hall Englewood Cliffs, 1993.
- [3] I. McCulloh, K.M. Carley, Detecting change in longitudinal social networks, *J. Soc. Struct.* 12 (3) (2011) 1–37.
- [4] L. Peel, A. Clauset, Detecting change points in the large-scale structure of evolving networks, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29, 2015.
- [5] E. Winter, The shapley value, *Handb. Game Theory Econ. Appl.* 3 (2002) 2025–2054.
- [6] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [7] S.-S. Ho, T.T. Kairamkonda, Change point detection in evolving graph using martingale, in: *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, 2024, pp. 466–473.
- [8] E. Page, Cumulative sum charts, *Technometrics* 3 (1) (1961) 1–9.
- [9] S. Roberts, Control chart tests based on geometric moving averages, *Technometrics* 1 (3) (1959) 239–250.
- [10] C.C. Aggarwal, Y. Li, P.S. Yu, On supervised change detection in graph streams, in: *Proceedings of the 2020 SIAM International Conference on Data Mining*, SIAM, 2020, pp. 289–297.
- [11] D. Koutra, N. Shah, J.T. Vogelstein, B. Gallagher, C. Faloutsos, DeltaCon: Principled massive-graph similarity function with attribution, *ACM Trans. Knowl. Discov. Data* 10 (3) (2016) 1–43.
- [12] J. Flossdorf, R. Fried, C. Jentsch, Online monitoring of dynamic networks using flexible multivariate control charts, *Soc. Netw. Anal. Min.* 13 (1) (2023) 87.
- [13] D. Grattarola, D. Zambon, L. Livi, C. Alippi, Change detection in graph streams by learning graph embeddings on constant-curvature manifolds, *IEEE Trans. Neural Netw. Learn. Syst.* 31 (6) (2019) 1856–1869.
- [14] A. Ferrari, C. Richard, Non-parametric community change-points detection in streaming graph signals, in: *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing*, ICASSP, IEEE, 2020, pp. 5545–5549.
- [15] S. Huang, Y. Hitti, G. Rabusseau, R. Rabbany, Laplacian change point detection for dynamic graphs, in: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 349–358.
- [16] Y. Xie, W. Wang, M. Shao, T. Li, Y. Yu, Multi-view change point detection in dynamic networks, *Inform. Sci.* 629 (2023) 344–357.
- [17] Y.L. Kei, J. Li, H. Li, Y. Chen, O.H. Madrid Padilla, Change point detection in dynamic graphs with decoder-only latent space model, 2025, arXiv preprint arXiv:2404.04719.
- [18] F. Fumagalli, M. Muschalik, E. Hüllermeier, B. Hammer, Incremental permutation feature importance (iPFI): towards online explanations on data streams, *Mach. Learn.* 112 (12) (2023) 4863–4903.
- [19] H. Choi, D. Kim, J. Kim, J. Kim, P. Kang, Explainable anomaly detection framework for predictive maintenance in manufacturing systems, *Appl. Soft Comput.* 125 (2022) 109147.
- [20] Z. Han, P. Chen, Y. Ma, V. Tresp, Explainable subgraph reasoning for forecasting on temporal knowledge graphs, in: *International Conference on Learning Representations*, 2020.
- [21] J. Tang, L. Xia, C. Huang, Explainable spatio-temporal graph neural networks, in: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 2432–2441.
- [22] V. Vovk, A. Gammerman, G. Shafer, *Algorithmic Learning in a Random World*, Springer-Verlag, Berlin, Heidelberg, 2005.
- [23] I. Nourtdinov, Conformal test martingales for hypergraphical models, *Ann. Math. Artif. Intell.* (2024).
- [24] I.-Y. Tsaknaki, F. Lillo, P. Mazzarisi, Bayesian autoregressive online change-point detection with time-varying parameters, *Commun. Nonlinear Sci. Numer. Simul.* 142 (2025) 108500.
- [25] D. Sulem, H. Kenlay, M. Cucuringu, X. Dong, Graph similarity learning for change-point detection in dynamic networks, *Mach. Learn.* 113 (1) (2024) 1–44.
- [26] F. Hinder, V. Vaquet, B. Hammer, One or two things we know about concept drift—a survey on monitoring in evolving environments. Part a: detecting concept drift, *Front. Artif. Intell.* 7 (2024).
- [27] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [28] W.L. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [29] C.D.T. Barros, M.R.F. Mendonça, A.B. Vieira, A. Ziviani, A survey on embedding dynamic graphs, *ACM Comput. Surv.* 55 (1) (2021) 1–37.
- [30] S.-S. Ho, A martingale framework for concept change detection in time-varying data streams, in: *Proceedings of the 22nd International Conference on Machine Learning*, 2005, pp. 321–327.
- [31] S.-S. Ho, H. Wechsler, A martingale framework for detecting changes in data streams by testing exchangeability, *IEEE Trans. Pattern Anal. Mach. Intell.* 32 (12) (2010) 2113–2127.
- [32] V. Vovk, I. Nourtdinov, A. Gammerman, Testing exchangeability on-line, in: *Proceedings of the 20th International Conference on Machine Learning*, 2003, pp. 768–775.
- [33] D. Volkonskiy, E. Burnaev, I. Nourtdinov, A. Gammerman, V. Vovk, Inductive conformal martingales for change-point detection, in: *Conformal and Probabilistic Prediction and Applications*, PMLR, 2017, pp. 132–153.
- [34] L. Dai, M.-R. Bouguelia, Testing exchangeability with martingale for change-point detection, *Int. J. Ambient. Comput. Intell.* 12 (2) (2021) 1–20.
- [35] V. Fedorova, A. Gammerman, I. Nourtdinov, V. Vovk, Plug-in martingales for testing exchangeability on-line, 2012, arXiv preprint arXiv:1204.3251.
- [36] J. Ville, *Etude Critique de la Notion de Collectif*, Gauthier-Villars, Paris, 1939.

- [37] P. Erdős, A. Rényi, et al., On the evolution of random graphs, *Publ. Math. Inst. Hung. Acad. Sci.* 5 (1) (1960) 17–60.
- [38] A.-L. Barabási, R. Albert, Emergence of scaling in random networks, *Science* 286 (5439) (1999) 509–512.
- [39] M.E.J. Newman, D.J. Watts, Renormalization group analysis of the small-world network model, *Phys. Lett. A* 263 (4–6) (1999) 341–346.
- [40] A. Madan, M. Cebrian, S. Moturu, K. Farrahi, et al., Sensing the “health state” of a community, *IEEE Pervasive Comput.* 11 (4) (2011) 36–45.



Shen-Shyang Ho is a tenured professor the Department of Computer Science at Rowan University. He received the BS degree in mathematics and computational science from the National University of Singapore in 1999, and the MS and PhD degrees in computer science from George Mason University in 2003 and 2007, respectively. From August 2007 to August 2010, he was a NASA Postdoctoral Program (NPP) fellow affiliated to the Jet Propulsion Laboratory (JPL) and a postdoctoral scholar affiliated to the California Institute of Technology. He was a researcher at CfAR, UMIACS from June 2010 to December 2011. He was a tenure-track assistant professor at the Nanyang Technological University between January 2012 and July 2016 in Singapore. His research interests include learning from graph streams/sequences, cooperative inference, federated learning, conformal prediction, and privacy issues in machine learning.



Tarun Teja Kairamkonda received his Master of Science in Computer Science at Rowan University in 2024. He received his Bachelor's in Technology with Electrical and Electronics Engineering from Guru Nanak Institute of Technology in India. He is currently an adjunct teaching faculty in the Computer Science Department at Rowan University. He has been recognized with the Computer Science Outstanding Graduate Student Award in 2024.



Izhar Ali received his BS in Computer Science and BA in Mathematics from Rowan University in 2024. He is currently pursuing an MS in Data Science and serves as a Graduate Research Fellow at Rowan University's College of Science & Mathematics. From August 2021 to January 2024, he worked as an NLP student researcher at Bristol Myers Squibb. He has served as an AI Technical Manager at Privatize, Inc. since June 2021 and is currently advising the MS Cybersecurity Engineering program at the University of San Diego. He has been recognized with the Dean's Award for Academic Excellence as an Outstanding Senior (2024), the James M. Shafer Excellence in Mathematics Medallion (2024), and the Outstanding Graduate Student Award (2025). Izhar is the co-founder of adversys.ai, focusing on AI-driven cybersecurity solutions.